How data structures are implemented internally in python?

1. Set
2. Dict / DefaultDict
3. OrderedDict
4. Heapq
5. List
6. Deque

OrderedDict

class collections.**OrderedDict**([*items*])

Return an instance of a `dict` subclass that has methods specialized for rearranging dictionary order

popitem(*last=True*)

The `popitem()` method for ordered dictionaries returns and removes a (key, value) pair. The pairs are returned in LIFO order if *last* is true or FIFO order if false.

move_to_end(*key*, *last=True*)

Move an existing *key* to either end of an ordered dictionary. The item is moved to the right end if *last* is true (the default) or to the beginning if *last* is false. Raises `KeyError` if the *key* does not exist:

```
d = OrderedDict.fromkeys('abcde')
d.move_to_end('b')
''.join(d)
'acdeb'
d.move_to_end('b', last=False)
''.join(d)
'Bacde'
```

Useful in LRU Cache implementations

How does OrderedDict support deleting from either end and transferring the key to the other end?

In **CPython**, it is implemented in **C** as:

A hash table (normal dict) + a doubly linked list

This gives:

- **O(1)** lookup (via hash table)
- **O(1)** insertion order maintenance (via linked list)
- **O(1)** deletion while preserving order

The OrderedDict object

```
typedef struct {
    PyDictObject dict;          // underlying dict
    _ODictNode *od_first;       // head of linked list
    _ODictNode *od_last;        // tail of linked list
    Py_ssize_t od_state;        // mutation counter
} PyODictObject;


> It literally embeds a PyDictObject
> Order is maintained outside the hash table
```

The linked list node

```
typedef struct _odictnode {
    PyObject *key;
```

```
    Py_hash_t hash;
    struct _odictnode *next;
    struct _odictnode *prev;
} _ODictNode;
```

Each key in the dict has **exactly one node** in this doubly linked list.

How lookup works (critical detail)

Unlike a naive design, OrderedDict **does not store nodes inside dict values**.

Instead:

- The **dict maps key → node**
- The node contains the key + hash + pointers

So conceptually:

dict[key] → _ODictNode*

This avoids extra hash lookups.

Insertion flow

od[key] = value

1. Compute hash(key)
2. Check if key already exists in dict
3. If **new key**:

    - Allocate _ODictNode
    - Append node to tail of linked list
    - Insert key → node into dict
4. Store the **actual value** in a parallel structure managed by dict

Order is updated **only on first insertion**, not on overwrite.

Deletion flow

del od[key]

1. Lookup node via dict
2. Unlink node from doubly linked list
3. Remove entry from dict
4. Free node

All steps are **O(1)**.

Iteration (why it's fast)

Iteration **does not touch the hash table order** at all.

od_first → node → node → node → od_last

This is why:

- Iteration is stable
- Reverse iteration is trivial

- popitem(last=True) is O(1)

Move-to-end operation

This is the secret sauce behind LRU caches.

od.move_to_end(key, last=True)

Internally:

1. Find node in dict
2. Unlink it
3. Reinsert at head or tail

No rehashing. No copying.

Operations and their time complexities:-

Operation	Complexity	Explanation
`popitem(last=True)`	O(1)	Head/tail removal
`popitem(last=False)`	O(1)	Head removal

`move_to_end(key, last=True)`	O(1)	Unlink + relink node
`od[key] = value`	O(1) avg amortized	Hash lookup + append node if new
`od[key]`	O(1) avg amortized	Dict lookup
`del od[key]`	O(1) avg amortized	Dict delete + unlink node
`key in od`	O(1) avg amortized	Hash lookup
`len(od)`	O(1)	Stored size

HeapQ

This module provides an implementation of the heap queue algorithm, also known as the priority queue algorithm.

Min-heaps are binary trees for which every parent node has a value less than or equal to any of its children. We refer to this condition as the heap invariant.

Why do we need Heap queue?

Now that you know what a heap queue is, let's see why it's useful in real-world scenarios:

1. Provides an efficient way to implement priority queues using heaps.
2. Helps maintain a list in heap order with minimal code and high performance.
3. Useful in algorithms like Dijkstra's, Huffman encoding or any task requiring quick access to smallest element.
4. Offers functions like heapify(), heappush() and heappop() for efficient insertion and removal.
5. Ideal for managing sorted data dynamically without full sorting after each operation.

Function	Complexity	Why
`heappush(heap, x)`	O(log n)	Sift-up
`heappop(heap)`	O(log n)	Sift-down
`heapreplace(heap, x)`	O(log n)	Pop + push combined
`heappushpop(heap, x)`	O(log n) (best-case O(1))	Conditional sift
`heapify(x)`	O(n)	Floyd's algorithm

Example code:-

```
import heapq
li = [25, 20, 15, 30, 40]

# Convert the list into a heap
heapq.heapify(li)
```

```
print("Heap queue:", li)
```

How is it implemented internally?

Core design choice (very important)

A heap is just a Python list.

There is:

- no Heap class, no node objects, no tree pointers

```
heap = []
```

The heap invariant (min-heap):

```
heap[i] <= heap[2*i + 1]
heap[i] <= heap[2*i + 2]
```

That’s it.

Why an array-based heap?

Python design constraints:

- Object allocation is expensive
- Pointer chasing is cache-unfriendly
- Python comparisons are slow

So heapq is optimized to:

- Minimize comparisons
- Minimize assignments
- Allocate **nothing**

The only two real operations

Everything in heapq reduces to **two internal helpers**:

_siftdown → used when inserting

Moves an element **up** the tree.

_siftup → used when removing

Moves an element **down** the tree.

_siftdown (bubble up)

Called after inserting at the end.

Simplified logic (from heapq.py):

```
def _siftdown(heap, startpos, pos):
    newitem = heap[pos]
    while pos > startpos:
        parentpos = (pos - 1) >> 1
        parent = heap[parentpos]
        if newitem < parent:
```

```
            heap[pos] = parent
            pos = parentpos
        else:
            break
    heap[pos] = newitem
```

Key internal optimizations

- Uses >> 1 instead of / 2
- **Moves parents down**, inserts once
- Avoids repeated swapping

⏱ **O(log n)** comparisons, minimal writes.

_siftup (bubble down)

Used after popping the root.

```
def _siftup(heap, pos):
    endpos = len(heap)
    startpos = pos
    newitem = heap[pos]
    childpos = 2*pos + 1

    while childpos < endpos:
```

```
        rightpos = childpos + 1
        if rightpos < endpos and heap[rightpos] < heap[childpos]:
            childpos = rightpos
        heap[pos] = heap[childpos]
        pos = childpos
        childpos = 2*pos + 1

    heap[pos] = newitem
    _siftdown(heap, startpos, pos)
```

Why _siftdown again at the end?

This is a **classic CPython micro-optimization**:

- _siftup pushes the “hole” to a leaf
- _siftdown fixes ordering on the way up
- Fewer comparisons than swapping at every level

---

heappush

```
def heappush(heap, item):
    heap.append(item)
    _siftdown(heap, 0, len(heap) - 1)
```

That’s literally it.

---

heappop

```
def heappop(heap):
    lastelt = heap.pop()
    if heap:
        returnitem = heap[0]
        heap[0] = lastelt
        _siftup(heap, 0)
        return returnitem
    return lastelt
```

Important detail:

- Last element is popped first → avoids shifting the list
- Root is replaced, then fixed

Why heapify is O(n)

```
def heapify(x):
    for i in reversed(range(len(x)//2)):
        _siftup(x, i)
```

Explanation:

- Leaves are already heaps
- Internal nodes are fixed bottom-up
- Total work sums to **O(n)** (Floyd's algorithm)

How to implement max heap?

Use:

heapq.heappush(heap, -x), that is store and pop elements by their negative values

Set

A Set in Python is used to store a collection of items with the following properties.

- No duplicate elements. If try to insert the same item again, it overwrites previous one.
- An unordered collection. When we access all items, they are accessed without any specific order and we cannot access items using indexes as we do in lists.
- Internally use hashing that makes set efficient for search, insert and delete operations. It gives a major advantage over a list for problems with these operations.
- Mutable, meaning we can add or remove elements after their creation, the individual elements within the set cannot be changed directly.

Operation	Complexity (avg)	Worst	Notes

x in s	O(1)	O(n)	Hash lookup
s.add(x)	O(1)	O(n)	May resize
s.remove(x)	O(1)	O(n)	Raises KeyError
s.discard(x)	O(1)	O(n)	No error
len(s)	O(1)	—	Stored count
s.clear()	O(n)	—	Frees table
x not in s	O(1)	O(n)	Negated lookup

Operation	**Reason**
Indexing (`s[0]`)	No order
Slicing	No order

Stable iteration	Hash-table scan
Sorting in-place	Not meaningful

Core idea (very important)

> **A Python set is a hash table, not a tree, not a bitset.**

It is **very similar to `dict`**, but:

- stores **only keys** (no values)
- uses a **slightly simpler entry structure**
- is optimized for **membership testing**

High-level structure

```
typedef struct {
    PyObject_HEAD
    Py_ssize_t fill;      // active + dummy entries
    Py_ssize_t used;      // active entries only
    Py_ssize_t mask;      // table size - 1
    setentry *table;      // hash table
    Py_hash_t hash;       // for frozenset only
    Py_ssize_t finger;    // iteration state
} PySetObject;
```

What these mean

Field	Meaning
`used`	Number of live elements
`fill`	Live + deleted (“dummy”)
`mask`	`table_size - 1` (power of two)
`table`	Pointer to entry array

`finger`	Iterator progress
`hash`	Cached hash (frozenset only)

Entry structure

```
typedef struct {
    PyObject *key;
    Py_hash_t hash;
} setentry;
```

Special values:

- `key == NULL` → **empty slot**
- `key == dummy` → **deleted slot**

(`dummy` is a global singleton marker)

Hashing & indexing

Index calculation:

```
i = hash & mask;
```

Table size is always a **power of two**, so modulo is a cheap bitmask.

Collision resolution (THIS IS KEY)

Python sets use **open addressing with perturbation probing**
(not chaining, not linear probing).

Probe sequence

```
i = (i * 5 + 1 + perturb) & mask;
perturb >>= 5;
```

Why this design?

- Starts with hash bits
- Gradually mixes in higher bits
- Avoids clustering
- Extremely cache-friendly

This same scheme is used by **dict**.

Lookup operation (`x in set`)

Steps

1. Compute `hash(x)`
2. Compute initial index: `hash & mask`
3. Probe until:
    - matching hash + equality → **found**
    - empty slot → **not found**

Complexity

Case	Cost
Average	**O(1)**
Worst (pathological hashes)	**O(n)**

Python randomizes hashes to prevent DOS attacks.

Insertion (`set.add(x)`)

1. Lookup slot using probing
2. If key already exists → do nothing
3. Else:
    - Insert `(key, hash)`
    - Increment `used` and `fill`
4. Check load factor → resize if needed

Load factor rules

Resize when:

```
fill * 3 >= mask * 2   (~66%)
```

This keeps probe lengths short.

Deletion (`set.remove`, `discard`)

Deletion does **not** clear the slot.

Instead:

```
entry->key = dummy;
```

Why?

- Clearing breaks probe chains

- Dummy preserves search correctness

Too many dummies → triggers resize.

Resizing (rehashing)

When resizing:

- Allocate new table (2× or ½)
- Reinsert **only live keys**
- Drop dummy entries

Cost: **O(n)**, but amortized **O(1)** per operation.

Iteration

Iteration is a **linear scan over the table**:

```
for i in range(table_size):
    if entry[i].key is real:
        yield entry[i].key
```

Important consequences

- Order is **arbitrary**
- Mutation during iteration raises `RuntimeError`
- Iteration cost is **O(table_size)**, not `O(len(set))`

How do hash tables work?

What a hash table fundamentally is

A hash table implements a mapping:

```
key  →  value
```

with the goal of:

- O(1) average lookup
- O(1) average insert
- O(1) average delete

It achieves this by turning a key into an array index using a hash function.

Hash functions (the heart of everything)

A hash function maps a key to an integer:

```
h = hash(key)
```

Good hash functions aim for:

- Determinism: same key → same hash
- Uniformity: hashes spread evenly
- Speed: called constantly
- Avalanche effect: small key change → big hash change

Example

```
hash("cat") → 0x8f3a2c91
```

```
hash("dog") → 0xa12bc490
```

The hash is not the index yet.

From hash → table index

Hash tables store data in an array of size `N`.

To map a hash to an index:

```
index = hash % N
```

In real implementations (including Python):

```
index = hash & (N - 1)
```

This only works when `N` is a power of two, which is why most hash tables use sizes like:

```
8, 16, 32, 64, 128, ...
```

Bitwise AND is much faster than modulo.

The collision problem (inevitable)

Since:

- Hashes are finite

- Keys are unbounded

Collisions are guaranteed

```
hash(a) % N == hash(b) % N
```

So the real question is:

> How do we store multiple keys that want the same slot?

There are two major strategies.

Collision resolution strategies

A) Separate chaining

Each bucket holds a linked list (or tree):

```
index 3 → (k1,v1) → (k2,v2) → (k3,v3)
```

Simple, Pointer-heavy, Cache-unfriendly

Used in many textbooks, not used in Python.

B) Open addressing (Python uses this)

All entries live inside the array itself.

If a slot is taken, you probe for another slot.

```
index = h(key)

if slot full:

    try another index
```

No linked lists. No extra allocations.

This is much more cache-friendly.

Probing strategies (open addressing)

1. Linear probing

```
i, i+1, i+2, i+3, ...
```

Reasons to avoid :- Causes clustering, Performance collapses at high load

---

2. Quadratic probing

```
i + 1², i + 2², i + 3², ...
```

Better, but still fragile.

---

3. Perturbation probing (Python's choice)

Used by dict and set in CPython.

Formula (simplified):

```
i = hash & mask

perturb = hash


while slot not usable:

    i = (i * 5 + 1 + perturb) & mask

    perturb >>= 5
```

Why this is brilliant

- Starts with low hash bits
- Gradually mixes in high bits
- Avoids clustering
- Deterministic but well-distributed
- Extremely cache-friendly

What’s actually stored in a hash table slot

In Python-style tables, each slot contains:

```
(key, hash, value)   # dict

(key, hash)          # set
```

Plus special markers:

Slot state	Meaning
EMPTY	Never used
ACTIVE	Holds key
DUMMY	Deleted slot

DUMMY slots are crucial.

Lookup step-by-step (`key in table`)

1. Compute `hash(key)`
2. Compute initial index
3. Probe:
    - If slot EMPTY → not found
    - If slot ACTIVE:
        - hash matches?
        - keys equal?
    - Else → keep probing

Average probes ≈ 1–3 if table is not crowded.

Insertion (`table[key] = value`)

1. Lookup key
2. If found → overwrite value
3. Else:
    - Insert into EMPTY or DUMMY slot
    - Increase count
4. Check load factor → resize if needed

Deletion (the tricky part)

You cannot just clear a slot.

Why?

```
A collides → placed later

B collides → placed even later


If A is deleted and slot cleared,

lookup(B) will stop early → BUG
```

Solution: DUMMY markers

Instead of clearing:

```
slot = DUMMY
```

This preserves probe chains.

Too many DUMMY slots → rehash.

Load factor & resizing

Load factor = `used_slots / table_size`

Python keeps it below ~66%

When exceeded:

- Allocate new table (≈ 2× size)
- Reinsert all ACTIVE entries
- Drop DUMMY slots

Why resizing is amortized O(1)

Although resize is O(n):

- Happens rarely
- Cost is spread across many inserts

Why average time is O(1)

Expected probe length:

```
≈ 1 / (1 - load_factor)
```

At 66% load:

```
≈ 3 probes
```

So:

- Lookup → constant
- Insert → constant
- Delete → constant

Worst case O(n) exists but is mitigated.

Hash randomization (security)

Python randomizes hash seeds at startup.

This prevents attackers from crafting keys that all collide.

This stopped real-world DoS attacks.

Why hash tables beat trees in Python

Structure	Access	Cache	Allocations
Hash table	O(1)	Excellent	Minimal
Tree	O(log n)	Poor	Many
Linked list	O(n)	Poor	Many

Hash tables align perfectly with modern CPUs.

Why Python dict/set are world-class

Python's implementation is famous because it:

- Uses open addressing
- Uses perturbation probing
- Uses power-of-two sizing
- Minimizes memory
- Optimizes for real workloads

This is why Python dict/set often beat naive C++ hash maps.

Mental model

```
Table:
```

```
[ EMPTY | keyA | DUMMY | keyB | EMPTY | keyC ]
```

```
Lookup:

start → hop → hop → found
```

No chains. No pointers. Just math.

Summary

Hash tables work because:

1. Hash functions spread keys uniformly
2. Array indexing is constant time
3. Probing resolves collisions efficiently
4. Load factor control keeps probes short
5. Resizing is amortized
6. Cache locality dominates performance

What are Self-Balancing Binary Search Trees?

In computer science, a **self-balancing binary search tree** (BST) is any node-based binary search tree that automatically keeps its height (maximal number of levels below the root) small in the face of arbitrary item insertions and deletions.

Most operations on a binary search tree (BST) take time directly proportional to the height of the tree, so it is desirable to keep the height small. A binary tree with height *h* can contain at most $2^0+2^1+\cdots+2^h = 2^{h+1}-1$ nodes. It follows that for any tree with *n* nodes and height *h*:

$$n \le 2h+1-1$$

And that implies:

$$h \ge \lceil \log 2(n+1)-1 \rceil \ge \lfloor \log 2n \rfloor$$

.

In other words, the minimum height of a binary tree with *n* nodes is $\log_2(n)$, rounded down; that is,

$$\lfloor \log 2n \rfloor$$

However, the simplest algorithms for BST item insertion may yield a tree with height *n* in rather common situations. For example, when the items are inserted in sorted key order, the tree degenerates into a linked list with *n* nodes. The difference in performance between the two situations may be enormous: for example, when *n* = 1,000,000, the minimum height is

$$\lfloor \log 2(1{,}000{,}000) \rfloor = 19$$

Self-balancing binary trees solve this problem by performing transformations on the tree (such as tree rotations) at key insertion times, in order to keep the height proportional to $\log_2(n)$. Although a certain overhead is involved, it is not bigger than the always necessary lookup cost and may be justified by ensuring fast execution of all operations.

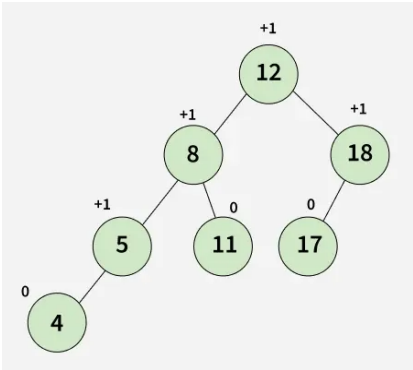
Language Implementations of self-balancing BST:

- set and map in C++ STL.
- TreeSet and TreeMap in Java. Most of the library implementations use Red Black Tree.
- Python standard library does not support Self Balancing BST. In Python, we can use bisect module to keep a set of sorted data. We can also use **PyPi** modules like rbtree (implementation of red-black tree) and pyavl (implementation of AVL tree).

Self-balancing binary search trees can be used in a natural way to construct and maintain ordered lists, such as priority queues. They can also be used for associative arrays; key-value pairs are simply inserted with an ordering based on the key alone. In this capacity, self-balancing BSTs have a number of advantages and disadvantages over their main competitor, hash tables. One advantage of self-balancing BSTs is that they allow fast (indeed, asymptotically optimal) enumeration of the items *in key order*, which hash tables do not provide. One disadvantage is that their lookup algorithms get more complicated when there may be multiple items with the same key. Self-balancing BSTs have better worst-case lookup performance than most[2] hash tables (O(logn) compared to O(n)), but have worse average-case performance (O(logn) compared to O(1)).

AVL Tree:

An AVL tree defined as a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees for any node cannot be more than one. The values above the nodes in the below diagram show difference between left and right subtrees.



Insertion in an AVL Tree follows the same basic rules as in a Binary Search Tree (BST):

- A new key is placed in its correct position based on BST rules (left < node < right).
- After the insertion, the **balance factor** of each node is checked during the path back up to the root. If any node becomes **unbalanced** (i.e., its balance factor becomes less than -1 or greater than +1), a **rotation** is required to restore the AVL property.

Insertion in AVL Tree:

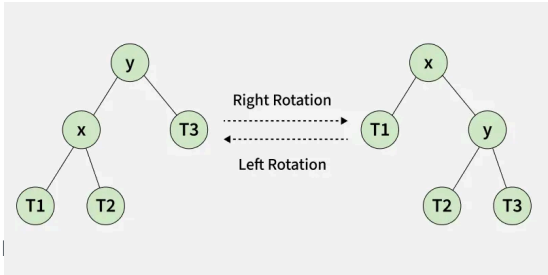
To make sure that the given tree remains AVL after every insertion, we must augment the standard BST insert operation to perform some re-balancing. Following are two basic operations that can be performed to balance a BST without violating the BST property (keys(left) < key(root) < keys(right)).

- Left Rotation
- Right Rotation

Approach: The idea is to use recursive BST insert, after insertion, we get pointers to all ancestors one by one in a bottom-up manner. So we don't need a parent pointer to travel up. The recursive code itself travels up and visits all the ancestors of the newly inserted node.

Follow the steps mentioned below to implement the idea:

- Perform the normal BST insertion.
- The current node must be one of the ancestors of the newly inserted node. Update the **height** of the current node.
- Get the balance factor (left subtree height - right subtree height) of the current node.
- If the balance factor is greater than **1,** then the current node is unbalanced and we are either in the **Left Left case** or **left Right case**. To check whether it is **left left case** or not, compare the newly inserted key with the key in the **left subtree root**.
- If the balance factor is less than **-1**, then the current node is unbalanced and we are either in the Right Right case or Right-Left case. To check whether it is the Right Right case or not, compare the newly inserted key with the key in the right subtree root.



Time Complexity: O(log

Auxiliary Space: O(logn), for recursion call stack as we have written a recursive method to insert

Steps to follow for deletion.
To make sure that the given tree remains AVL after every deletion, we must augment the

standard BST delete operation to perform some re-balancing. Following are two basic operations that can be performed to re-balance a BST without violating the BST property (keys(left) < key(root) < keys(right)).

1. Left Rotation
2. Right Rotation

https://www.geeksforgeeks.org/dsa/deletion-in-an-avl-tree/

Time Complexity: The rotation operations (left and right rotate) take constant time as only few pointers are being changed there. Updating the height and getting the balance factor also take constant time. So the time complexity of AVL delete remains same as BST delete which is O(h) where h is height of the tree. Since AVL tree is balanced, the height is O(log n). So time complexity of AVL delete is O(log n).